

LAPORAN PRAKTIKUM SISTEM OPERASI

MODUL 5 EKSPLORASI PROSES



Disusun Oleh :

Ilham Lii Assidaq 2311104068

Dosen Pengampu :

Yudha Islami Sulistya, S.Kom., M.Cs

PROGRAM STUDI S1 SOFTWARE ENGINEERING

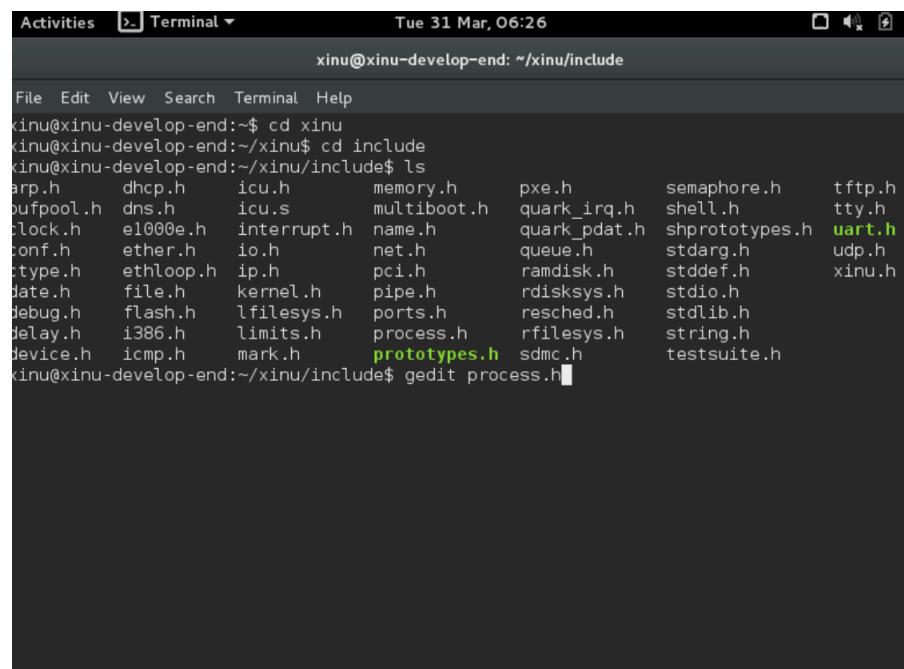
FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

1. Dasar Teori

A. Process.h

File `process.h` merupakan salah satu berkas header paling krusial dalam kernel sistem operasi Xinu yang berfungsi sebagai pusat pengaturan, definisi konstanta, dan struktur data untuk manajemen proses secara efisien. Di dalam berkas ini, diatur berbagai parameter penting seperti batasan jumlah proses maksimal yang aktif secara simultan (konstanta `NPROC`), status berkala dari setiap proses, hingga parameter awal pembuatan proses seperti alokasi ukuran stack dan prioritas kerja. Selain itu, terdapat definisi proses khusus seperti `NULLPROC` yang bertugas menjaga agar CPU tidak berada dalam kondisi benar-benar menganggur (`idle`), serta konstanta penanda khusus (`stack marker`) yang diletakkan di bagian atas memori guna mendeteksi terjadinya kebocoran memori atau `stack overflow`.



```
xinu@xinu-develop-end: ~/xinu/include
File Edit View Search Terminal Help
xinu@xinu-develop-end:~$ cd xinu
xinu@xinu-develop-end:~/xinu$ cd include
xinu@xinu-develop-end:~/xinu/include$ ls
arp.h      dhcp.h      icu.h        memory.h     pxe.h        semaphore.h  tftp.h
bufpool.h  dns.h       icu.s         multiboot.h  quark_irq.h  shell.h      tty.h
clock.h     el000e.h    interrupt.h   name.h        quark_pdat.h shprototypes.h  uart.h
conf.h      ether.h     io.h          net.h         queue.h       stdarg.h     stdio.h
ctype.h     ethloop.h  ip.h          pci.h         ramdisk.h     stddef.h     xinu.h
date.h      file.h      kernel.h      pipe.h        rdisksys.h    stdio.h
debug.h     flash.h    lfilesys.h    ports.h       resched.h     stdlib.h
delay.h     i386.h     limits.h      process.h     rfilesys.h    string.h
device.h    icmp.h     mark.h        prototypes.h  sdmc.h        testsuite.h
xinu@xinu-develop-end:~/xinu/include$ gedit process.h
```

Di samping konfigurasi konstanta sistem, berkas `process.h` juga mendefinisikan `struct procent`, sebuah struktur data inti penyusun tabel proses (`proctab[]`) yang menyimpan informasi komprehensif mengenai status, prioritas, serta alokasi perangkat dari setiap proses. Struktur ini dilengkapi dengan fungsi validasi `Process ID (PID)` untuk mencegah kesalahan akses terhadap entri proses yang tidak valid atau kosong, serta pembatasan jumlah `device descriptor` maksimal sebanyak lima buah per proses demi menjaga efisiensi penyejajaran memori sistem. Melalui kombinasi struktur data yang sederhana namun terorganisasi dengan baik pada halaman dokumen SO-LAPRAK ini, kernel Xinu mampu menjalankan fungsi-fungsi

kompleks seperti penjadwalan CPU, sinkronisasi, serta tata kelola sumber daya komputasi secara optimal.

B. Jumlah Device Descriptor

```
/* Number of device descriptors a process can have open */  
#define NDESC          5      /* Must be odd to make procent 4N bytes */
```

Sistem Xinu membatasi setiap proses untuk hanya dapat menghubungkan maksimal 5 device descriptor atau perangkat (seperti file virtual, UART, dan terminal). Alasan pemilihan angka lima ini diperjelas oleh catatan pada kode sumber, yaitu "Must be odd to make procent 4N bytes". Penggunaan jumlah yang ganjil ini bertujuan akademis agar ukuran keseluruhan dari struktur data proses (struct procent) memenuhi prinsip memory alignment atau berada pada batas kelipatan 4 byte.

C. Struktur Tabel Proses (struct procent)

Struktur struct procent adalah inti dari manajemen proses dalam Xinu.

Setiap entri di tabel proses (proctab[]) berisi informasi lengkap mengenai status,

```
/* Definition of the process table (multiple of 32 bits) */  
  
struct procent {  
    uint16 prstate;      /* Entry in the process table */  
    pri16  prprio;      /* Process state: PR_CURR, etc. */  
    char   *prstkptr;    /* Process priority */  
    char   *prstkbase;   /* Saved stack pointer */  
    uint32 prstklen;     /* Base of run time stack */  
    char   prname[PNMLEN]; /* Stack length in bytes */  
    sid32  prsem;        /* Process name */  
    pid32  prparent;     /* Semaphore on which process waits */  
    umsg32 prmsg;        /* ID of the creating process */  
    bool8  prhasmsg;     /* Message sent to this process */  
    int16  prdesc[NDESC]; /* Nonzero iff msg is valid */  
    /* Device descriptors for process */  
};
```

prioritas, stack, dan perangkat yang digunakan oleh proses. Dengan desain sederhana ini, Xinu dapat melakukan penjadwalan, komunikasi antarproses, serta pengelolaan sumber daya dengan efisien.

D. Struktur Global dan File Implementasi Proses di Xinu

Pada bagian penutup berkas process.h, diatur sejumlah variabel global beserta definisi ekstra yang memegang peranan krusial dalam mekanisme tata kelola proses di kernel Xinu, yaitu:

- Marker Stack (Deteksi Overflow)
- Variabel Global untuk Manajemen Proses

E. Implementasi Fungsi Proses

```
xinu@xinu-develop-end: ~/xinu/system
```

File	Edit	View	Search	Terminal	Help
arp.h		dhcp.h	icu.h	memory.h	
bufpool.h		dns.h	icu.s	multiboot.h	
clock.h		e1000e.h	interrupt.h	name.h	
conf.h		ether.h	io.h	net.h	
ctype.h		ethloop.h	ip.h	pci.h	
date.h		file.h	kernel.h	pipe.h	
debug.h		flash.h	lfilesys.h	ports.h	
delay.h		i386.h	limits.h	process.h	
device.h		icmp.h	mark.h	prototypes.h	
xinu@xinu-develop-end:~/xinu/include\$ gedit process.h					
xinu@xinu-develop-end:~/xinu/include\$ cd ..					
xinu@xinu-develop-end:~/xinu\$ cd system					
xinu@xinu-develop-end:~/xinu/system\$ ls					
ascd.c		freebuf.c	init.c	open.c	ready.c
bufinit.c		freemem.c	initialize.c	panic.c	receive.c
chprio.c		getbuf.c	insert.c	pci.c	recvclr.c
clkdisp.S		getc.c	insrt.c	ptclear.c	recvtime.c
clkhandler.c		getdev.c	intr.S	ptcount.c	resched.c
clkinit.c		getitem.c	ioerr.c	ptcreate.c	resume.c
close.c		getmem.c	ionull.c	ptdelete.c	seek.c
conf.c		getpid.c	kill.c	ptinit.c	semcount.c
control.c		getprio.c	kprintf.c	ptrecv.c	semcreate.c
create.c		getstk.c	main.c	ptreset.c	semdelete.c
ctxsw.S		getticks.c	mark.c	ptsend.c	semreset.c
debug.c		gettime.c	meminit.c	putc.c	send.c
dev.c		getutime.c	mkbufpool.c	queue.c	signal.c
exit.c		i386.c	newqueue.c	read.c	signaln.c
xinu@xinu-develop-end:~/xinu/system\$					

Folder ini berisi file-file sumber C (.c) yang mengimplementasikan fungsi-fungsi sistem operasi terkait proses, seperti:

- `create.c` Membuat proses baru
- `resume.c` Melanjutkan proses yang di-suspend
- `sleep.c` Menidurkan proses untuk jangka waktu tertentu
- `ready.c` Memasukkan proses ke antrean siap jalan
- `resched.c` Mengatur penjadwalan proses (scheduler)
- `kill.c` Menghapus atau menghentikan proses
- `semcreate.c` / `semctl.c` Mengatur semaphore
- `send.c` / `receive.c` Mengatur komunikasi antarproses
- `wait.c` / `signal.c` Sinkronisasi antarproses

2. Manfaat

Melalui praktikum ini, saya dapat memahami struktur dan fungsi Process Control Block (PCB) pada sistem operasi Xinu sehingga saya mengetahui bagaimana sistem mengelola informasi dari setiap proses secara terstruktur. Selain itu, saya juga memperoleh kemampuan untuk menganalisis dan memodifikasi proses pada sistem Xinu secara sederhana untuk meningkatkan pemahaman saya terhadap manajemen proses, sekaligus mengembangkan

keterampilan praktis dalam mendalami cara kerja sistem operasi, khususnya dalam hal pengelolaan serta eksekusi proses.

3. Jurnal

A. [10 Poin] Jawablah pertanyaan berikut ini:

a. Berapa banyaknya maksimum proses yang ada pada Xinu?

NPROC = 8 proses

b. Berapa maksimal panjang nama suatu proses pada Xinu?

16 karakter

```
/* Miscellaneous process definitions */  
  
#define PNMLEN      16      /* Length of process "name"      */  
#define NULLPROC    0      /* ID of the null process        */
```

c. Berapa nilai prioritas awal pada saat proses dibuat?

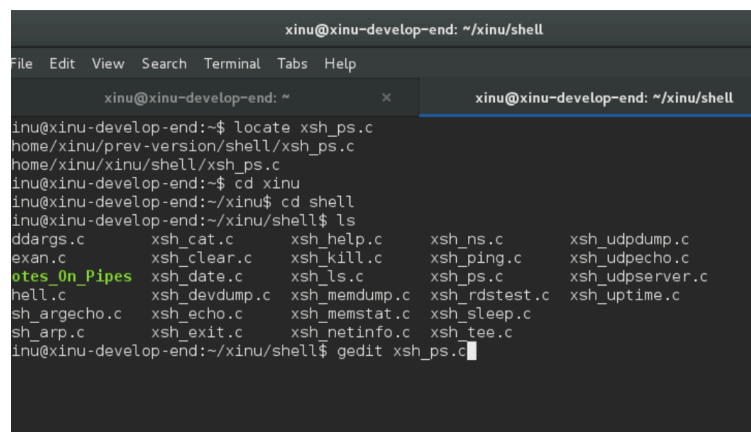
20

d. Ada berapa total state pada Xinu? Sebutkan!

8 State

```
/* Process state constants */  
  
#define PR_FREE      0      /* Process table entry is unused */  
#define PR_CURR      1      /* Process is currently running  */  
#define PR_READY     2      /* Process is on ready queue     */  
#define PR_RECV      3      /* Process waiting for message   */  
#define PR_SLEEP     4      /* Process is sleeping           */  
#define PR_SUSP      5      /* Process is suspended          */  
#define PR_WAIT      6      /* Process is on semaphore queue */  
#define PR_RECTIM    7      /* Process is receiving with timeout */
```

B. [20 Poin] Perintah ps adalah perintah untuk menampilkan statistik process yang berjalan. Source code dari ps tersimpan pada file xsh_ps.c. Carilah file tersebut dan beri komentar pada 20 baris terakhir di source code tersebut!



```
xinu@xinu-develop-end: ~/xinu/shell  
File Edit View Search Terminal Tabs Help  
xinu@xinu-develop-end: ~  
xinu@xinu-develop-end:~$ locate xsh_ps.c  
home/xinu/prev-version/shell/xsh_ps.c  
home/xinu/xinu/shell/xsh_ps.c  
xinu@xinu-develop-end:~$ cd xinu  
xinu@xinu-develop-end:~/xinu$ cd shell  
xinu@xinu-develop-end:~/xinu/shell$ ls  
ddargs.c      xsh_cat.c      xsh_help.c      xsh_ns.c      xsh_udpdump.c  
exan.c        xsh_clear.c    xsh_kill.c      xsh_ping.c    xsh_udpecho.c  
otes_On Pipes xsh_date.c     xsh_ls.c        xsh_ps.c      xsh_udpserver.c  
hell.c        xsh_devdump.c  xsh_memdump.c   xsh_rdstest.c xsh_uptime.c  
sh_argecho.c  xsh_echo.c     xsh_memstat.c   xsh_sleep.c  
sh_arp.c      xsh_exit.c     xsh_netinfo.c   xsh_tee.c  
xinu@xinu-develop-end:~/xinu/shell$ gedit xsh_ps.c
```

```

Open ▾ xsh - /xinu/shell Save
/* Print header for items from the process table */

printf("%3s %-16s %5s %4s %4s %10s %-10s %10s %3s %7s\n",
       "Pid", "Name", "State", "Prio", "Ppid", "Stack Base",
       "Stack Ptr", "Stack Size", "Msg", "Content");

printf("%3s %-16s %5s %4s %4s %10s %-10s %10s %3s %7s\n",
       "-----", "-----", "-----", "-----", "-----", "-----",
       "-----", "-----", "-----", "-----");

/* Output information for each process */

/* Nama : Ilham Lii | , Nim : 2311104068 */

for (i = 0; i < NPROC; i++) {
    prptr = &proctab[i];
    if (prptr->prstate == PR_FREE) { /* Skip unused slots */
        continue;
    }
    printf("%3d %-16s %s %4d %4d 0x%08X 0x%08X %8d %3d %7d\n",
           i, prptr->prname, pstate[(int)prptr->prstate],
           prptr->prprio, prptr->prparent, prptr->prstkbase,
           prptr->prstkptr, prptr->prstklen,
           prptr->prhasmsg,
           (prptr->prhasmsg ? (int)prptr->prmsg : 0));
}
return 0;

```

- C. [35 Poin] Ubahlah perintah `ps` (source code: `xsh_ps.c`) pada Xinu sehingga menampilkan informasi tambahan berupa kolom yang berisi total message yang ada pada proses seperti gambar di bawah ini:

```

xsh $ ps
Pid Name                State Prio Ppid Stack Base Stack Ptr  Stack Size Msg Content
-----
0 prnull                ready  0    0  0x005FDFFC 0x00FFFF04  8192  1    4
1 rdsproc                susp  200   0  0x005FBFFC 0x005FBFC8 16384  0    0
2 ipout                  wait  500   0  0x005F7FFC 0x005F7F40  8192  0    0
3 netin                  wait  500   0  0x005F5FFC 0x005F5EE0  8192  0    0
5 Main process           recv  20    4  0x005E3FFC 0x005E3F64 65536  0    0
6 shell                  recv  50    5  0x005D3FFC 0x005D3C7C  8192  0    0
7 ps                     curr  20    6  0x005F3FFC 0x005F3DC8  8192  0    0

```

```

Activities ▾ xsh - /xinu/shell xsh_ps.c Save
/* Output information for each process */
/* Menambahkan informasi Msg (prmsg) dan Content (prmsg) pada setiap proses */

for (i = 0; i < NPROC; i++) {
    prptr = &proctab[i];
    if (prptr->prstate == PR_FREE) { /* Skip unused slots */
        continue;
    }

    /*Menampilkan baris proses beserta jumlah pesan (prmsg)*/
    /*dan isi pesan (prmsg) dari Process Control Block (PCB) */

    printf("%3d %-16s %s %4d %4d 0x%08X 0x%08X %8d %3d %7d\n",
           i,
           prptr->prname,
           pstate[(int)prptr->prstate],
           prptr->prprio,
           prptr->prparent,
           prptr->prstkbase,
           prptr->prstkptr,
           prptr->prstklen,
           prptr->prhasmsg,
           prptr->prmsg);
}
return 0;

```

```
Welcome to Xinu!

xsh $ uptime
Xinu has been up 0 minute(s)
xsh $ ps
Pid Name          State Prio Ppid Stack Base Stack Ptr  Stack Size Msg Content
-----
0 prnull          ready  0    0 0x005FDFFC 0x00FFFE4 8192 1 3
1 ipout           wait   500  0 0x005FBFFC 0x005FBF30 8192 0 0
2 netin           wait   500  0 0x005F9FFC 0x005F9ED0 8192 0 0
4 Main process    recv   20   3 0x005E7FFC 0x005E7F34 65536 0 0
5 shell           recv   50   4 0x005F7FFC 0x005F79AC 8192 0 6
7 ps              curr   20   5 0x005F5FFC 0x005F5FC4 8192 0 0
xsh $
```

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:35 | ttyS0

D. [35 Poin] Ubahlah perintah uptime pada Xinu sehingga menampilkan lamanya Xinu

```
Welcome to Xinu!

xsh $ uptime
Xinu has been up 0 minute(s)
xsh $
```

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | V

sejak booting hanya dalam satuan menit.

4. Kesimpulan

Berdasarkan praktikum yang telah saya laksanakan pada modul ini, saya dapat menyimpulkan bahwa saya telah berhasil mendalami tata kelola proses pada sistem operasi Xinu melalui pemahaman struktur dan fungsi Process Control Block (PCB) yang didefinisikan di dalam berkas process.h. Melalui analisis langsung terhadap komponen kernel tersebut, termasuk batasan jumlah proses, validasi Process ID (PID), alokasi memori stack, hingga mekanisme deteksi overflow, saya kini mampu menganalisis serta melakukan modifikasi sederhana pada proses sistem. Pengalaman teknis ini tidak hanya memperkuat pemahaman teoritis saya mengenai manajemen proses tingkat rendah, tetapi juga secara nyata meningkatkan keterampilan praktis saya dalam memahami mekanisme eksekusi internal yang berjalan di dalam sebuah sistem operasi.